BK-CI 技术方案分享

流水线大变量（>4K）技术方案

通过“短引用 + 溢出表 + 按需加载”，把流水线变量单值能力从 4K 提升到 4M，同时保障历史列表、启动参数、日志和归档链路的稳定性。

4M
单变量硬上限

0 扩主表
不 ALTER 现网大表

按需加载
${{ }} 访问真实值

闭环
归档迁移与清理

适用范围：研发 / 测试 / 前端 / 运维 / 技术评审

版本：2026-08

目录

> 本文是对外分享版，重点解释“为什么要做、整体怎么做、关键链路如何保证稳定、上线如何验收”。详细实现可继续参考代码库中的分册设计文档。

01 背景与目标

02 总体架构图

03 数据模型

04 写入流程

05 读取与表达式求值

06 前端与 API 交互

07 归档迁移与清理

08 上线与验收

核心结论

存得下

大变量真实值写入溢出表，主表只保存短引用 `__BK_OVF__:<len>`。

读得稳

列表、历史、启动参数 Tab 默认不还原真实值，避免 N × 4M 的内存风险。

用得准

只有 `${{ xxx }}` 按需加载真实值，`$xxx` / `${xxx}` 保持引用串。

1. 背景与目标

现状痛点

- `T_PIPELINE_BUILD_VAR.VALUE` 限制 4000 字符，超过后无法可靠入库。
- 启动参数完整 JSON 写入 `BUILD_PARAMETERS`，多个大值会接近 mediumtext 上限。
- 历史列表批量反序列化大 JSON，存在 OOM 风险。
- Worker / 日志直接打印大文本，会拖垮日志和前端展示。
- 归档库若不迁移溢出表，历史大变量真实值会丢失。

目标能力

- 变量单值上限从 4K 提升至 **4M**。
- 主表不扩列、不存截断明文，只存纯引用串。
- 表达式、单 key 查询、retry / replay 才按需还原真实值。
- 前端默认只展示"大变量约 N 字符"，点击后再查真实值。
- 归档库可查、可迁、可清，不留下孤儿数据。

范围	改造前	改造后
≤4K 变量	主表保存真实值	保持不变，完全兼容
4K ~ 4M 变量	失败、丢弃或截断风险	溢出表保存真实值，主表保存引用串
>4M 变量	行为不统一	拒绝 / 丢弃并记录告警
历史列表	可能加载大 JSON	只看到短引用，稳定分页

2. 总体架构图

> 大变量方案的核心是把“默认读取”和“真实值读取”拆开：默认链路永远轻量，只有明确需要真实值的链路才按 key 加载。

总体架构：短引用 + 双溢出表 + 按需加载

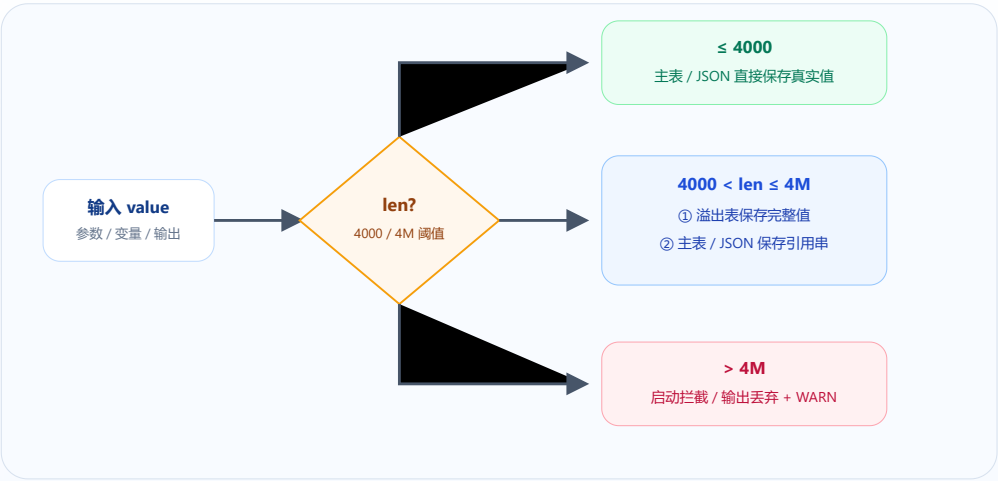


载体	表	生命周期	用途
运行期变量	`T_PIPELINE_BUILD_VAR_OVERFLOW`	短期，线上库可按月分区清理	构建执行、表达式、插件输出
启动参数快照	`T_PIPELINE_BUILD_HISTORY_PARAM_OVERFLOW`	长期，与构建历史同寿	启动参数 Tab、retry、replay

3. 写入流程

写入链路遵循"先写真实值，再写引用串"的顺序，避免主表出现引用但溢出表没有真实值。

写入流程：按长度分流



写入侧关键点

运行期变量

入口包括插件输出、脚本变量、`setVariable`、批量写变量。大值先写`T_PIPELINE_BUILD_VAR_OVERFLOW`，主表再写引用串。

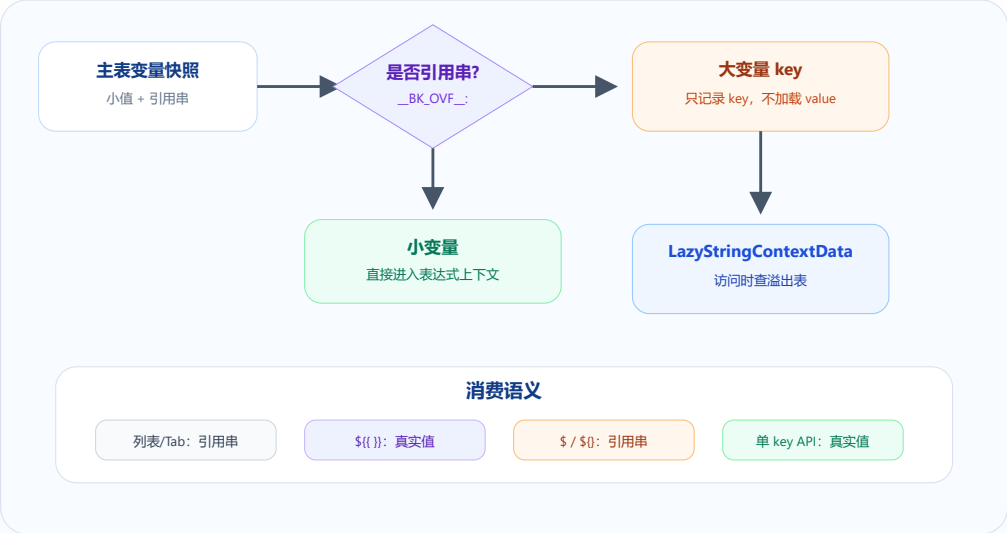
启动参数

写`BUILD_PARAMETERS`前生成引用化副本，真实值落`T_PIPELINE_BUILD_HISTORY_PARAM_OVERFLOW`，不原地修改运行期变量对象。

4. 读取与表达式流程

> 读取侧的原则是：默认不还原，单 key 才还原；表达式只有双花括号 `${{ xxx }}` 能拿真实值。

读取流程：默认轻量，访问时懒加载

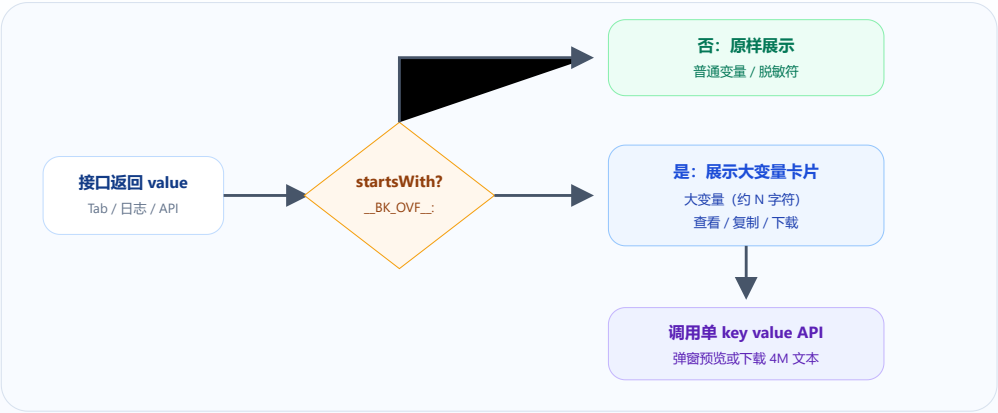


场景	行为	是否加载真实值
历史列表 / 启动参数 Tab	返回引用串	否
`${{ BIG }}`	按需查溢出表并替换真实值	是
`$BIG` / `${BIG}`	替换为引用串	否
日志点击查看 / 参数单 key 查看	单 key API 返回真实值	是，单 key
retry / replay	完整还原启动参数	是，受总量上限保护

5. 前端与用户交互

前端不再把引用串当普通文本展示，而是识别为“大变量占位”，用户点击后再查真实值。

前端交互流程：识别引用串，点击后单 key 查询



前端场景	改造要求
启动参数 Tab	识别 `__BK_OVF__:`，展示“大变量（约 N 字符）”，点击调用 `parameters/value`
插件输出日志	识别 `output(overflow): key=__BK_OVF__:N`，点击调用 `variables/value`
设置页	隐藏 `failIfVariableInvalid` 相关开关和文案
参数校验	取消前端 4K 拦截，与后端 4M 单值上限对齐

6. 归档迁移与清理

> 大变量不是只要“当前构建可用”就结束，还必须保证归档库可查询、迁移过程可控、过期清理不残留孤儿数据。

归档生命周期：变量主表 + 两张溢出表一起迁移和清理

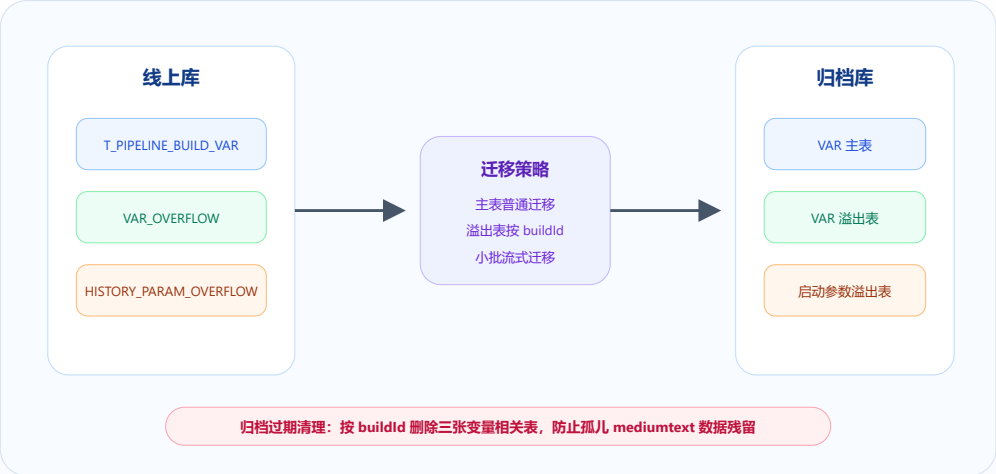


表	线上库	归档库
`T_PIPELINE_BUILD_VAR`	按既有开关与构建清理策略删除	归档过期时删除
`T_PIPELINE_BUILD_VAR_OVERFLOW`	随变量主表级联删除，也可 DROP 分区	随变量主表删除，不做月分区
`T_PIPELINE_BUILD_HISTORY_PARAM_OVERFLOW`	随构建历史按 buildId 删除	随归档历史按 buildId 删除

7. 上线方案与验收

上线顺序



冒烟检查清单

功能

- >4K 且 ≤4M 变量主表为引用串，溢出表有完整值。
- `${{ BIG }}` 得真实值，`$BIG` 得引用串。
- 启动参数列表只返回引用，单 key 可查真实值。
- retry / replay 可完整还原大启动参数。

稳定性

- 历史列表分页查询无明显内存增长。
- 日志出现 `output(overflow)` 且可点击查询。
- 归档后 `archiveFlag=true` 可查真实值。
- 归档过期清理后三张表无孤儿数据。

回滚策略

应用服务可直接回滚；DDL 不需要回滚。旧服务会把 `__BK_OVF__:<len>` 当普通短字符串处理，因此构建可继续流转，但大变量真实值不会被表达式还原。

8. 总结

这套方案的价值： 在不改现网变量主表结构的前提下，把大变量能力提升到 4M，并通过引用串、按需加载、预算保护、单 key 查询和归档闭环，控制了存储、读取、前端和运维风险。

对用户

可以传入和使用 4K 以上的大变量，明确通过 `${{ xxx }}` 获取真实值。

对系统

列表与历史默认保持小响应，避免因为大文本导致接口、内存和前端展示不稳定。

对运维

线上库可分区清理，归档库随历史生命周期清理，迁移按小批流式处理。

参考文档	用途
`PIPELINE_LARGE_VARIABLE_TECHNICAL_DESIGN.md`	完整技术总览
`PIPELINE_BUILD_VAR_OVERFLOW_DESIGN.md`	运行期变量溢出细节
`PIPELINE_BUILD_STARTUP_PARAM_OVERFLOW_DESIGN.md`	启动参数历史溢出细节
`FRONTEND_LARGE_VARIABLE_INTEGRATION.md`	前端对接与交互说明
`ARCHIVE_BUILD_VARIABLE_MIGRATION_DESIGN.md`	归档迁移与清理方案

本文档为对外分享版，图片为内嵌 SVG，可直接打印或导出 PDF。